

Web Application Development

Lecture 4: State with useState

SWE 381 – Web Application Development

Dr. Ohoud Alharbi Dr. Achraf Gazdar

Department of Software Engineering
College of Computer and Information Sciences
King Saud University

Term 2 – 2025/2026

Learning Objectives

By the end of this lecture, you will be able to:

- Explain why **regular variables** cannot drive reactive UIs in React
- Import and use the useState Hook with correct syntax
- Describe the **state update cycle** and what triggers a re-render
- Apply the rule of **never mutating state directly**
- Use useState with **all data types**: strings, numbers, booleans, objects, arrays
- Manage **multiple state variables** in one component
- Distinguish between **props** and **state** and know when to use each
- Compute **derived values** from state without storing redundant state

Agenda

- 1 Why Regular Variables Fail
- 2 useState Syntax & Destructuring
- 3 State Update Cycle
- 4 Never Mutate State Directly
- 5 All Data Types with useState
- 6 Multiple State Variables
- 7 Props vs State
- 8 Derived Values
- 9 Recap & Summary

Outline

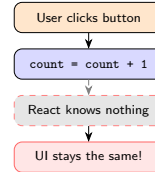
- 1 Why Regular Variables Fail
- 2 useState Syntax & Destructuring
- 3 State Update Cycle
- 4 Never Mutate State Directly
- 5 All Data Types with useState
- 6 Multiple State Variables
- 7 Props vs State
- 8 Derived Values
- 9 Recap & Summary

The Problem – Regular Variables Don't Update the UI

```
1 // Attempt to use a regular variable
2 function Counter() {
3   let count = 0; // regular variable
4
5   function handleClick() {
6     count = count + 1; // variable changes...
7     console.log(count); // ...prints 1, 2, 3
8   }
9
10  return (
11    <div>
12      { /* BUT the screen never updates! */ }
13      <h1>Count: {count}</h1>
14      <button onClick={handleClick}>
15        Increment
16      </button>
17    </div>
18  );
19 }
```

⚠ The screen always shows 0

The variable increments in memory but React **never re-renders** because it has no way of knowing the variable changed.



Two things are required:

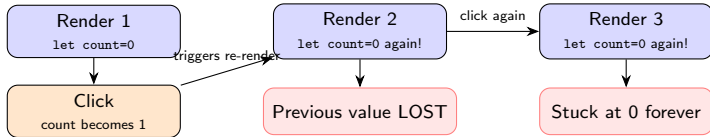
- 1 Retain data between renders
- 2 Trigger React to re-render with new data

Solution: `useState` Hook

Why Re-render Doesn't Help Regular Variables

React rendering behaviour

When React re-renders a component, it calls the function again from scratch. Any **local variable** (e.g. `let count = 0`) is **re-initialised to its default** every render. The previous value is lost.



useState solves both problems

`useState` stores its value **outside the function** (in React's memory), so the value **persists** across re-renders. And calling the setter **triggers** a fresh render automatically.

Outline

- 1 Why Regular Variables Fail
- 2 **useState Syntax & Destructuring**
- 3 State Update Cycle
- 4 Never Mutate State Directly
- 5 All Data Types with useState
- 6 Multiple State Variables
- 7 Props vs State
- 8 Derived Values
- 9 Recap & Summary

W3Schools – React useState Hook

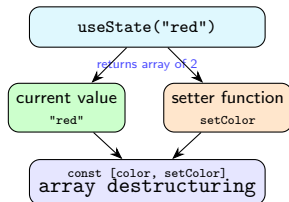
The React useState Hook allows us to track **state** in a function component. State generally refers to **data or properties** that need to be tracked in an application. useState accepts an **initial state** and returns **two values**: the current state, and a function that updates the state.

Step-by-step setup:

```
// Step 1: import useState (named export)
import { useState } from 'react';

function FavoriteColor() {
  // Step 2: declare state variable
  // [currentValue, setterFunction]
  const [color, setColor] = useState("red");
  //      ^           ^
  // state var   setter fn   initial value

  return (
    <h1>My favorite color is {color}!</h1>
  );
}
```



Naming Convention and Updating State

W3Schools naming convention:

```
1 // Pattern: [name, setName]
2 const [color, setColor] = useState("red");
3 const [count, setCount] = useState(0);
4 const [isOpen, setIsOpen] = useState(false);
5 const [user, setUser] = useState(null);
6 const [items, setItems] = useState([]);
```

Full W3Schools example – color buttons:

```
1 import { useState } from 'react';
2
3 function FavoriteColor() {
4   const [color, setColor] = useState("red");
5
6   return (
7     <>
8       <h1>My favorite color is {color}!</h1>
9       <button onClick={() => setColor("blue")}>
10         Blue
11       </button>
12       <button onClick={() => setColor("red")}>
13         Red
14       </button>
15       <button onClick={() => setColor("pink")}>
16         Pink
17       </button>
18     </>
19   );
20 }
```

Initial render

My favorite color is **red**!

Blue

Red

Pink

After clicking Blue

My favorite color is **blue**!

Blue

Red

Pink



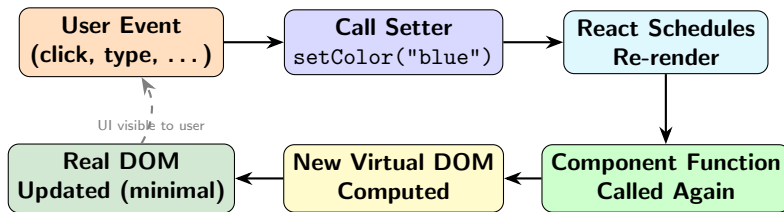
Convention

The setter is always named **set** + **variable name**. Using any other name works, but following the convention makes code self-explanatory.

Outline

- 1 Why Regular Variables Fail
- 2 useState Syntax & Destructuring
- 3 State Update Cycle**
- 4 Never Mutate State Directly
- 5 All Data Types with useState
- 6 Multiple State Variables
- 7 Props vs State
- 8 Derived Values
- 9 Recap & Summary

The State Update Cycle



State is a snapshot

Each render has its **own snapshot** of state. The value does not change mid-render; only the **next** render sees the new value.



Batching

React may **batch** multiple `setState` calls into a single re-render for performance. You always receive the latest state on the next render.

State Update – Counter Example

```
1 import { useState } from 'react';
2
3 function Counter() {
4   // Initial state = 0
5   const [count, setCount] = useState(0);
6
7   return (
8     <div>
9       <h1>Count: {count}</h1>
10
11       /* Each click calls setter -> re-render */
12       <button onClick={() =>
13         setCount(count + 1)
14       }>
15         + Increment
16       </button>
17
18       <button onClick={() =>
19         setCount(count - 1)
20       }>
21         - Decrement
22       </button>
23
24       <button onClick={() =>
25         setCount(0)
26       }>
27         Reset
28       </button>
29     </div>
30   );
31 }
```

After 3 increments

Count: 3

+ Increment

- Decrement

Reset

What happens on each click:

- 1 `setCount(count + 1)` is called
- 2 React schedules a re-render
- 3 `Counter()` runs again
- 4 `count` now holds the new value
- 5 JSX is re-evaluated and DOM updated

Key insight

`count` inside any given render is **constant**. It only changes for the next render.

Functional Updater Form – Safer Updates

W3Schools – best practice for derived updates

When the new state value **depends on the previous** value, pass a **function** to the setter. React guarantees it receives the latest state, even when updates are batched.

Direct update (can be stale):

```
// May be stale if React batches calls
setCount(count + 1);
setCount(count + 1);
// Both see the SAME old count
// Result: only increments by 1!
```

Functional updater (always fresh):

```
// Receives the latest state each time
setCount(prev => prev + 1);
setCount(prev => prev + 1);
// Each call sees the result of the last
// Result: correctly increments by 2
```

Practical example:

```
1 function StepCounter() {
2   const [steps, setSteps] = useState(0);
3
4   function increment() {
5     // Safe even if called rapidly
6     setSteps(prev => prev + 1);
7   }
8
9   return (
10    <div>
11      <p>Today: {steps} steps!</p>
12      <button onClick={increment}>
13        I took a step
14      </button>
15    </div>
16  );
17 }
```

Outline

- 1 Why Regular Variables Fail
- 2 useState Syntax & Destructuring
- 3 State Update Cycle
- 4 Never Mutate State Directly**
- 5 All Data Types with useState
- 6 Multiple State Variables
- 7 Props vs State
- 8 Derived Values
- 9 Recap & Summary

The Golden Rule – Never Mutate State Directly

W3Schools – React useState Hook

We should **never directly update state**. For example: `color = "blue"` is not allowed. We must always use the setter function returned by `useState`.

Wrong – direct mutation:

```
1 const [color, setColor] = useState("red");
2
3 // WRONG: direct assignment
4 color = "blue";           // Error!
5
6 // WRONG: mutating an object field
7 user.name = "Ahmed";      // no re-render!
8
9 // WRONG: pushing into an array
10 items.push(newItem);      // no re-render!
```



Why mutation breaks React

React uses **reference equality** to detect changes. If the reference stays the same (direct mutation), React thinks nothing changed and **skips the re-render**.

Correct – always use the setter:

```
1 const [color, setColor] = useState("red");
2
3 // CORRECT: call setter
4 setColor("blue");
5
6 // CORRECT: new object via spread
7 setUser({ ...user, name: "Ahmed" });
8
9 // CORRECT: new array via spread
10 setItems([...items, newItem]);
11
12 // CORRECT: new array via filter
13 setItems(items.filter(i => i.id !== id));
```

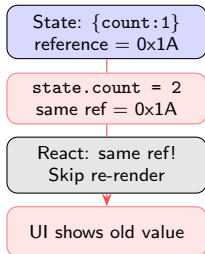


Memory model

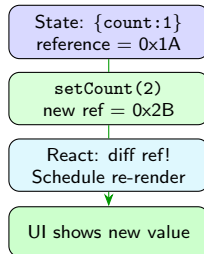
Treat state as **immutable**. Always create a **new** value and hand it to the setter. React then swaps the old value for the new one.

Why Direct Mutation Silently Breaks Things

WRONG: mutate directly



CORRECT: use setter



Outline

- 1 Why Regular Variables Fail
- 2 useState Syntax & Destructuring
- 3 State Update Cycle
- 4 Never Mutate State Directly
- 5 All Data Types with useState**
- 6 Multiple State Variables
- 7 Props vs State
- 8 Derived Values
- 9 Recap & Summary

useState with Strings and Numbers

W3Schools – React useState Hook

The useState Hook can be used to keep track of **strings**, **numbers**, **booleans**, **arrays**, **objects**, and any combination of these!

String state:

```
1 import { useState } from 'react';
2
3 function FavoriteColor() {
4   const [color, setColor] = useState("red");
5
6   return (
7     <>
8       <h1>Favorite: {color}</h1>
9       <button onClick={() => setColor("blue")}>
10         Blue
11       </button>
12       <button onClick={() => setColor("green")}>
13         Green
14       </button>
15     </>
16   );
17 }
```

Number state:

```
1 function ClickCounter() {
2   const [count, setCount] = useState(0);
3
4   return (
5     <div>
6       <p>Clicked {count} times</p>
7       <button onClick={() =>
8         setCount(prev => prev + 1)
9       }>
10         Click me
11       </button>
12     </div>
13   );
14 }
```

After 3 clicks

Clicked 3 times

Click me

useState with Booleans – Toggle Pattern

Boolean toggle:

```
1 import { useState } from 'react';
2
3 function DarkModeToggle() {
4   const [isDark, setIsDark] = useState(false);
5
6   return (
7     <div style={{
8       background: isDark ? '#1a1a2e' : '#fff',
9       color:      isDark ? '#fff'      : '#000',
10      padding: '20px'
11    }}>
12       <h2>
13         {isDark ? 'Dark Mode' : 'Light Mode'}
14       </h2>
15       <button onClick={() =>
16         setIsDark(prev => !prev)
17       }>
18         Toggle Mode
19       </button>
20     </div>
21   );
22 }
```

isDark = false

Light Mode

Toggle Mode

isDark = true

Dark Mode

Toggle Mode

💡 Boolean toggle idiom

`setIsDark(prev => !prev)` is the standard toggle pattern. It always flips the current value correctly.

useState with Objects – Use Spread to Update

W3Schools – useState with Objects

When state is an object, use the **spread operator** to copy the existing fields and override only the one(s) you want to change. Do **not** mutate the object directly.

```
1 import { useState } from 'react';
2
3 function StudentProfile() {
4   const [student, setStudent] = useState({
5     name: "Ahmed",
6     gpa: 3.8,
7     dept: "SWE"
8   });
9
10  // Spread existing fields, override only 'name'
11  const changeName = () => {
12    setStudent({
13      ...student, // keep gpa and dept
14      name: "Sara" // update only name
15    });
16  };
17
18  return (
19    <div>
20      <p>Name: {student.name}</p>
21      <p>GPA: {student.gpa}</p>
22      <p>Dept: {student.dept}</p>
23      <button onClick={changeName}>
24        Change Name
25      </button>
26    </div>
27  );
28 }
```

Before click

Name: Ahmed
GPA: 3.8
Dept: SWE

Change Name

After click

Name: Sara
GPA: 3.8
Dept: SWE

Change Name

⚠ Never do this

`student.name = "Sara"` mutates directly – React will not re-render and the UI stays stale.

useState with Arrays – Add, Remove, Update

```
1 import { useState } from 'react';
2
3 function TodoList() {
4   const [todos, setTodos] = useState([
5     { id: 1, text: "Learn React" },
6     { id: 2, text: "Build a project" },
7   ]);
8   const [input, setInput] = useState("");
9
10  // ADD: spread old array + new item
11  const addTodo = () => {
12    setTodos([
13      ...todos,
14      { id: Date.now(), text: input }
15    ]);
16    setInput("");
17  };
18
19  // REMOVE: filter out the item
20  const removeTodo = (id) => {
21    setTodos(todos.filter(t => t.id !== id));
22  };
23
24  return (
25    <div>
26      <input value={input}
27        onChange={e => setInput(e.target.value)}
28        placeholder="New task..." />
29      <button onClick={addTodo}>Add</button>
30      <ul>
31        {todos.map(t => (
32          <li key={t.id}>
33            {t.text}
34            <button onClick={() =>
35              removeTodo(t.id)
36            }> X </button>
37          </li>
38        ))}
39      </ul>
40    </div>
41  );
42}
```

Live to-do list

New task... Add

- Learn React X
- Build a project X

Array operations cheatsheet:

Action	Pattern
Add item	[...arr, newItem]
Remove item	arr.filter(...)
Update item	arr.map(...)
push()	NEVER – mutates!
splice()	NEVER – mutates!

Updating an Object Inside an Array

Pattern: map + spread

To update one object inside a state array: use `map()` to iterate, and the **spread operator** to clone and patch the matching item.

```
1 const [students, setStudents] = useState([
2   { id: 1, name: "Ahmed", gpa: 3.5 },
3   { id: 2, name: "Sara", gpa: 3.8 },
4   { id: 3, name: "Khalid", gpa: 3.2 },
5 ]);
6
7 // Update Sara's GPA to 4.0
8 const updateGpa = (targetId, newGpa) => {
9   setStudents(
10     students.map(s =>
11       s.id === targetId
12         ? { ...s, gpa: newGpa } // new object for the match
13         : s // unchanged objects returned as-is
14     )
15   );
16 };
17
18 // Call: updateGpa(2, 4.0)
19 // Result: [ {id:1, name:"Ahmed", gpa:3.5},
20 //           {id:2, name:"Sara", gpa:4.0}, <-- updated
21 //           {id:3, name:"Khalid", gpa:3.2} ]
```

💡 The map + spread pattern

`map()` creates a new array. For the matching item, `{...s, gpa: newGpa}` creates a new object copy with the field overridden. **No original data is mutated.**

Outline

- 1 Why Regular Variables Fail
- 2 useState Syntax & Destructuring
- 3 State Update Cycle
- 4 Never Mutate State Directly
- 5 All Data Types with useState
- 6 Multiple State Variables**
- 7 Props vs State
- 8 Derived Values
- 9 Recap & Summary

Multiple State Variables – W3Schools

W3Schools – React useState Hook

We could create **multiple state Hooks** to track individual values. Or we can just use one state and include an **object**.

Multiple separate state variables:

```
1 import { useState } from 'react';
2
3 function Car() {
4   // Each piece of state is independent
5   const [brand, setBrand] = useState("Ford");
6   const [model, setModel] = useState("Mustang");
7   const [year, setYear] = useState(1964);
8   const [color, setColor] = useState("red");
9
10  return (
11    <div>
12      <h1>My {brand}!</h1>
13      <p>
14        It is a {color} {model} from {year}.
15      </p>
16      <button onClick={() => setColor("blue")}>
17        Paint Blue
18      </button>
19    </div>
20  );
21 }
```

Output

My Ford!

It is a **red** Mustang from 1964.

Paint Blue

Rules for multiple useState calls:

- Must be called at the **top level** of the component
- **Never** inside loops, conditions, or nested functions
- React identifies them **by order of declaration**
- Updating one state does **not** affect the others

One Object State vs Multiple Variables

Multiple variables (recommended):

```
1 // Easy to update one without spreading
2 const [brand, setBrand] = useState("Ford");
3 const [color, setColor] = useState("red");
4 const [year, setYear] = useState(1964);
5
6 // Updating color is simple
7 setColor("blue");
```

One object state (alternative):

```
1 const [car, setCar] = useState({
2   brand: "Ford",
3   color: "red",
4   year: 1964
5 });
6
7 // Must spread all fields when updating one
8 setCar({ ...car, color: "blue" });
```

Aspect	Multiple	One obj
Updating one field	Simple	Needs spread
Related data together	No	Yes
Readability	Clear	Grouped
Best for	Unrelated data	Related data

💡 W3Schools guidance

Use multiple variables for **unrelated** pieces of data. Group into an object when the fields always **change together** (e.g., form fields, position coordinates).

⚠️ Hook order must be stable

React tracks hooks by **call order**. Never put `useState` inside an `if` block or a loop – this breaks the order and causes crashes.

Multiple States – Practical Form Example

```
1 import { useState } from 'react';
2
3 function RegistrationForm() {
4   // Separate state for each field
5   const [name, setName] = useState("");
6   const [email, setEmail] = useState("");
7   const [password, setPassword] = useState("");
8   const [agree, setAgree] = useState(false);
9
10  function handleSubmit(e) {
11    e.preventDefault();
12    if (!agree) { alert("Please accept the terms!"); return; }
13    alert(`Registered: ${name} (${email})`);
14  }
15
16  return (
17    <form onSubmit={handleSubmit}>
18      <input placeholder="Full name" value={name}
19        onChange={e => setName(e.target.value)} />
20      <input placeholder="Email" value={email}
21        onChange={e => setEmail(e.target.value)} />
22      <input placeholder="Password" type="password" value={password}
23        onChange={e => setPassword(e.target.value)} />
24      <label>
25        <input type="checkbox" checked={agree}
26          onChange={e => setAgree(e.target.checked)} />
27        I accept the terms
28      </label>
29      <button type="submit">Register</button>
30      { /* Live preview */ }
31      <p>Name preview: <b>{name}</b></p>
32    </form>
33  );
34 }
```

Outline

- 1 Why Regular Variables Fail
- 2 useState Syntax & Destructuring
- 3 State Update Cycle
- 4 Never Mutate State Directly
- 5 All Data Types with useState
- 6 Multiple State Variables
- 7 Props vs State**
- 8 Derived Values
- 9 Recap & Summary

Props vs State – Key Differences

Feature	Props	State
Who owns it	Parent component	The component itself
Who can change it	Parent (read-only to child)	The component (setter)
How passed	As JSX attributes	Declared with <code>useState</code>
Triggers re-render	When parent re-renders	When setter is called
Purpose	Customise the component	Track data that changes
Initial source	Passed from parent	Defined inside component
Analogy	Function argument	Local variable + memory

Props

Think of props as a **photograph** – a snapshot passed in. The child reads it but never alters it.

State

Think of state as the component's **memory** – it remembers things between renders and can change over time.

Props vs State – Side-by-Side Code

Props – from parent, read-only:

```
1 // Parent passes data via props
2 function App() {
3   return (
4     <CourseCard
5       title="Web App Dev"
6       credits={3}
7     />
8   );
9 }
10
11 // Child reads props -- never modifies them
12 function CourseCard({ title, credits }) {
13   return (
14     <div>
15       <h2>{title}</h2>
16       <p>{credits} credits</p>
17     </div>
18   );
19 }
20 // props.title = "other" -- WRONG!
```

State – internal, mutable:

```
1 import { useState } from 'react';
2
3 // Component manages its own state
4 function LikeButton() {
5   // Starts at 0, only this
6   // component can change it
7   const [likes, setLikes] = useState(0);
8
9   return (
10     <button onClick={() =>
11       setLikes(prev => prev + 1)
12     }>
13       Likes: {likes}
14     </button>
15   );
16 }
17 // Each <LikeButton /> has its
18 // own independent count!
```



How to decide

Ask: “Does this data come from outside the component?” Yes → props. “Does this data change inside the component?” Yes → state.

State is Isolated – Each Instance is Independent

W3Schools – Component instances

When you use a component multiple times, **each instance has its own state**. Clicking a button on one component does not affect the others.

```
1 import { useState } from 'react';
2
3 function Counter({ label }) {
4   // Each Counter has its OWN count
5   const [count, setCount] = useState(0);
6   return (
7     <div style={{ border: '1px solid',
8       padding: '8px', margin: '4px' }}>
9       <p>{label}: {count}</p>
10      <button onClick={() =>
11        setCount(prev => prev + 1)
12      }>
13        +1
14      </button>
15    </div>
16  );
17 }
18
19 function App() {
20   return (
21     <>
22       <Counter label="Apples" />
23       <Counter label="Bananas" />
24       <Counter label="Cherries" />
25     </>
26   );
27 }
```

After some clicks

Apples: 3

+1

Bananas: 1

+1

Cherries: 0

+1

💡 State is local

Each `<Counter />` instance lives in its own memory. They share the same component logic but have completely **independent state values**.

Initialising State from Props

Pattern: prop as initial state

A prop can be used as the **initial value** of a state variable. The component then manages the state independently. Changes to the prop after the first render do **not** update the state automatically.

```
1 import { useState } from 'react';
2
3 function EditableLabel({ initialText }) {
4   // Prop only sets the STARTING value
5   const [text, setText] = useState(initialText);
6   const [editing, setEditing] = useState(false);
7
8   return editing ? (
9     <input
10       value={text}
11       onChange={e => setText(e.target.value)}
12       onBlur={() => setEditing(false)}
13       autoFocus
14     />
15   ) : (
16     <span onClick={() => setEditing(true)}>
17       {text} (click to edit)
18     </span>
19   );
20 }
21
22 // Usage
23 function App() {
24   return (
25     <EditableLabel initialText="SWE 381" />
26   );
27 }
```

Before click

SWE 381 (click to edit)

While editing

SWE 381

💡 Naming convention

Prefix the prop with **initial** (e.g. `initialText`, `initialCount`) to signal it is only the starting value, not a controlled value.

Outline

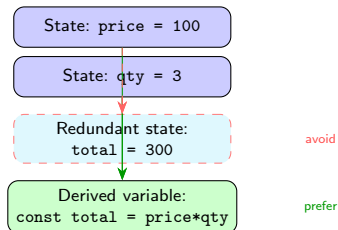
- 1 Why Regular Variables Fail
- 2 useState Syntax & Destructuring
- 3 State Update Cycle
- 4 Never Mutate State Directly
- 5 All Data Types with useState
- 6 Multiple State Variables
- 7 Props vs State
- 8 Derived Values**
- 9 Recap & Summary

What Are Derived Values?

Derived values are values that can be **computed from existing state or props**. They should **not** be stored as separate state.

Why avoid redundant state?

- Two pieces of state describing the same thing can become **out of sync**
- More `setState` calls = harder to maintain
- Makes debugging confusing



Golden rule

If a value can be **computed from state or props during render**, do not put it in state. Just compute it as a regular **JavaScript variable** inside the component body.

Derived Values in Practice

```
1 import { useState } from 'react';
2
3 function ShoppingCart() {
4   const [items, setItems] = useState([
5     { id: 1, name: "React Book", price: 45, qty: 2 },
6     { id: 2, name: "Laptop Stand", price: 30, qty: 1 },
7     { id: 3, name: "USB Hub", price: 20, qty: 3 },
8   ]);
9
10  // DERIVED: computed from state each render
11  // No need for separate state!
12  const total = items.reduce(
13    (sum, i) => sum + i.price * i.qty, 0
14  );
15  const itemCount = items.reduce(
16    (sum, i) => sum + i.qty, 0
17  );
18  const hasItems = items.length > 0;
19
20  return (
21    <div>
22      <h2>Cart ({itemCount} items)</h2>
23      {hasItems
24        ? items.map(i => (
25          <p key={i.id}>
26            {i.name}: ${i.price} x {i.qty}
27          </p>
28        ))
29        : <p>Your cart is empty.</p>
30      }
31      <h3>Total: ${total}</h3>
32    </div>
33  );
34 }
```

Browser Output

Cart (6 items)

React Book: \$45 x 2

Laptop Stand: \$30 x 1

USB Hub: \$20 x 3

Total: \$180

All three are derived:

- total – computed by reduce
- itemCount – sum of quantities
- hasItems – boolean from length

💡 Computed on every render

These values are recalculated every time items state changes. They are **always in sync** automatically – no extra `setState` needed.

More Derived Value Examples

```
1 import { useState } from 'react';
2
3 function StudentGradeTracker() {
4   const [scores, setScores] = useState([
5     85, 92, 78, 95, 88
6   ]);
7
8   // All DERIVED -- no extra state!
9   const average = scores.length > 0
10     ? scores.reduce((a,b) => a+b, 0)
11       / scores.length
12     : 0;
13
14   const highest = Math.max(...scores);
15   const lowest = Math.min(...scores);
16   const passed = scores.filter(s => s >= 60);
17   const passRate = scores.length > 0
18     ? Math.round(
19       (passed.length / scores.length) * 100
20     )
21     : 0;
22   const grade = average >= 90 ? 'A'
23     : average >= 80 ? 'B'
24     : average >= 70 ? 'C' : 'D';
25
26   return (
27     <div>
28       <h2>Grade Tracker</h2>
29       <p>Average: {average.toFixed(1)}</p>
30       <p>Grade: {grade}</p>
31       <p>Highest: {highest}</p>
32       <p>Lowest: {lowest}</p>
33       <p>Pass Rate: {passRate}%</p>
34     </div>
35   );
36 }
```

Browser Output

Grade Tracker

Average: 87.6

Grade: B

Highest: 95

Lowest: 78

Pass Rate: 100%

All five values derived from one state array scores:

- average – sum / length
- grade – ternary chain
- highest/lowest – Math methods
- passRate – filter + ratio

⚠ What NOT to do

Do not store average, grade, etc. as separate useState variables. They would need to be manually kept in sync every time scores changes.

Outline

- 1 Why Regular Variables Fail
- 2 useState Syntax & Destructuring
- 3 State Update Cycle
- 4 Never Mutate State Directly
- 5 All Data Types with useState
- 6 Multiple State Variables
- 7 Props vs State
- 8 Derived Values
- 9 Recap & Summary

Putting It All Together – Complete Example

```
1 import { useState } from 'react';
2
3 function CourseManager() {
4   // Multiple state variables
5   const [courses, setCourses] = useState([
6     { id: 1, title: "Web App Dev", credits: 3, active: true },
7     { id: 2, title: "SW Design", credits: 3, active: false },
8     { id: 3, title: "AI Basics", credits: 2, active: true },
9   ]);
10  const [filter, setFilter] = useState("all"); // "all" | "active"
11
12  // DERIVED: no extra state needed
13  const displayed = filter === "active"
14    ? courses.filter(c => c.active)
15    : courses;
16
17  const totalCredits = displayed.reduce((sum, c) => sum + c.credits, 0);
18
19  // Toggle active flag -- immutable update
20  const toggleActive = (id) => {
21    setCourses(courses.map(c =>
22      c.id === id ? { ...c, active: !c.active } : c
23    ));
24  };
25
26  return (
27    <div>
28      <h2>Courses ({totalCredits} credits shown)</h2>
29      <button onClick={() => setFilter("all")}>All</button>
30      <button onClick={() => setFilter("active")}>Active Only</button>
31      <ul>
32        {displayed.map(c => (
33          <li key={c.id} style={{ color: c.active ? "green" : "gray" }}>
34            {c.title} ({c.credits} cr)
35            <button onClick={() => toggleActive(c.id)}>Toggle</button>
36          </li>
37        ))}
38      </ul>
39    </div>
40  );
41 }
```

Lecture 4 – Key Takeaways

- 1 **Regular variables** are re-initialised every render and cannot trigger UI updates
- 2 `useState` stores state **outside the function** in React's memory and triggers re-renders when changed
- 3 **Syntax:** `const [value, setValue] = useState(initialValue)` – always destructure
- 4 Calling the **setter** schedules a re-render; the UI updates on the *next* render
- 5 Use `setValue(prev => prev + 1)` when new state **depends on current** state
- 6 **Never mutate** state directly – always produce a new value (spread for objects/arrays)
- 7 `useState` works with **all data types**: strings, numbers, booleans, objects, arrays
- 8 **Multiple** `useState` calls are fine; call them only at the **top level** of the component
- 9 **Props** come from parent (read-only); **state** is internal (mutable via setter)
- 10 **Derived values** should be computed as regular variables, not stored as extra state

Next Lecture: `useEffect` – Side Effects, Lifecycle & Data Fetching

Practice Exercises

- 1 Build a `Counter` component with Increment, Decrement, and Reset buttons using `useState`. Observe the re-render by adding a `console.log` inside the component body.
- 2 Create a `FavoriteColor` component (W3Schools example) with four colour buttons. Use `useState("red")` as the initial state.
- 3 Build a `DarkModeToggle` that switches the background and text colours using a boolean state. Use the `prev => !prev` functional updater.
- 4 Create a `StudentCard` component that holds a student object in state (`name`, `gpa`, `dept`). Add a button that updates only the `gpa` field using the spread operator.
- 5 Build a `TodoList` where the user can add items via an input and remove them by clicking X. Use an array state; never use `push` or `splice`.
- 6 Add a live **derived counter** to your to-do list showing “*N tasks remaining*” without storing the count as separate state.
- 7 **Challenge:** Create a `ShoppingCart` component: items array in state, quantity controls (+ / -) per item, and a derived `total` price. Never store `total` as state – compute it on every render.